

# AI Obstacle Detection System for the Visually Impaired

Tudor – Nicolae Ursu  
Alexandru Pavelescu  
Ioana Rizea

## Table of Contents

|                                               |    |
|-----------------------------------------------|----|
| Table of Contents .....                       | 2  |
| 1. Introduction .....                         | 4  |
| 1.1 Problem Statement .....                   | 4  |
| 1.2 Proposed Solution .....                   | 4  |
| 1.3 Objectives .....                          | 4  |
| 2. Background & Related Work .....            | 5  |
| 2.1 Object Detection Overview .....           | 5  |
| 2.2 YOLO Architecture .....                   | 5  |
| 2.3 Assistive Technology Context .....        | 5  |
| 2.4 Roboflow Platform .....                   | 5  |
| 3. Methodology .....                          | 6  |
| 3.1 Data Collection .....                     | 6  |
| 3.2 Data Integration via Roboflow .....       | 6  |
| 3.3 Class Mapping .....                       | 6  |
| 3.4 Model Training .....                      | 7  |
| 3.5 Model Persistence .....                   | 7  |
| 3.6 Inference & Validation .....              | 7  |
| 4. System Architecture .....                  | 8  |
| 4.1 Overview .....                            | 8  |
| 4.2 Back-End: FastAPI Prediction Server ..... | 8  |
| 4.3 Tunnel: ngrok .....                       | 8  |
| 4.4 Front-End: Browser Client .....           | 8  |
| 4.5 System Diagram .....                      | 9  |
| 5. Implementation Details .....               | 10 |
| 5.1 Development Environment .....             | 10 |
| 5.2 Dataset Download & Preparation .....      | 10 |
| 5.3 Model Training Code .....                 | 10 |
| 5.4 FastAPI Prediction Endpoint .....         | 10 |
| 5.5 Front-End Frame Capture Loop .....        | 11 |
| 6. Alternatives Considered .....              | 12 |
| 6.1 Detection Model Alternatives .....        | 12 |
| 6.2 Training Environment Alternatives .....   | 12 |
| 7. Results & Evaluation .....                 | 13 |
| 7.1 Training Performance .....                | 13 |
| 7.2 Qualitative Validation .....              | 13 |
| 7.3 Real-Time Performance .....               | 13 |
| 7.4 Limitations .....                         | 13 |
| 8. Discussion .....                           | 15 |

|                               |    |
|-------------------------------|----|
| 8.1 Design Decisions .....    | 15 |
| 8.2 Current Limitations ..... | 15 |
| 8.3 Future Work.....          | 15 |
| 9. Conclusion .....           | 16 |
| References.....               | 17 |

# 1. Introduction

Visual impairment affects hundreds of millions of people worldwide, significantly limiting their ability to navigate everyday environments safely and independently. Current assistive technologies like white canes, guide dogs, and basic smartphone apps provide limited situational awareness and often fail to detect hazards at sufficient distance and speed for effective avoidance.

This project addresses that gap by developing a real-time obstacle detection system powered by a custom-trained deep learning model. The system uses a smartphone camera to stream video, a YOLOv8 object detection model to identify hazards, and text-to-speech audio feedback to alert the user — all with sub-second latency.

## 1.1 Problem Statement

Visually impaired individuals face significant risks when navigating urban environments. Obstacles such as potholes, open manholes, motorcycles, poles, and curbs can cause serious injury yet are entirely invisible to the user without an assistive system. Existing solutions lack the accuracy or real-time capability required for safe navigation.

## 1.2 Proposed Solution

Our system combines three key components:

- A custom YOLOv8n model trained on obstacle-specific datasets
- A FastAPI backend server hosted on Google Colab and tunnelled via ngrok
- A lightweight browser-based HTML/JavaScript front-end capturing live camera frames

When an obstacle is detected, the system draws a bounding box around it on the camera view and announces the obstacle type via the browser's Web Speech API, providing an immediate audio alert to the user.

## 1.3 Objectives

- Collect and integrate multi-source datasets covering road hazards and urban obstacles
- Train a compact, high-accuracy YOLOv8 model optimised for mobile inference
- Build a real-time REST API capable of processing camera frames at 1 fps with low latency
- Deploy a mobile-friendly front-end with visual bounding boxes and voice alerts
- Evaluate system performance on validation data and real-world test scenarios

## 2. Background & Related Work

### 2.1 Object Detection Overview

Object detection is a core computer vision task that combines image classification with spatial localisation. Modern approaches use single-stage detectors, which directly predict class probabilities and bounding box offsets from a single pass through a convolutional neural network, enabling real-time performance.

### 2.2 YOLO Architecture

YOLO (You Only Look Once) is a family of single-stage detectors known for their speed and accuracy trade-off. YOLOv8, released by Ultralytics, introduces a new anchor-free detection head, an improved CSPDarknet backbone, and a path aggregation neck that enhances multi-scale feature fusion. The nano variant (YOLOv8n) is optimised for edge and mobile deployment with a minimal parameter count while retaining strong detection performance.

Key architectural improvements in YOLOv8 over prior versions include:

- Anchor-free detection: eliminates manual anchor tuning
- Decoupled head: separate branches for classification and regression
- Mosaic augmentation: combines four training images for richer context
- C2f module: enhanced cross-stage partial connections

### 2.3 Assistive Technology Context

Prior work on assistive navigation systems has explored depth sensors (e.g. Microsoft Kinect), LiDAR, and stereo cameras. While accurate, these hardware-dependent approaches are costly and impractical for everyday carry. Smartphone-based solutions using standard RGB cameras have gained traction due to their accessibility. Projects such as Microsoft Seeing AI and various academic prototypes demonstrate the viability of vision-based assistance, but are typically limited to object recognition rather than hazard detection.

### 2.4 Roboflow Platform

Roboflow is a computer vision platform that streamlines dataset management, annotation, augmentation, and export. In this project it was used to consolidate two independently sourced Kaggle datasets into a single unified training set with consistent labelling, augmentation pipelines, and a standardised YOLOv8 export format.

## 3. Methodology

### 3.1 Data Collection

Two publicly available Kaggle datasets were selected to provide broad coverage of the target obstacle classes:

- **Dataset A — Nayan Beale Obstacles Detection (YOLO):** A diverse dataset containing images of urban obstacles including poles, benches, trash bins, motorcycles, persons, fences, doors, trees, flowerpots, stairs, and crosswalks. Images cover indoor and outdoor environments with varying lighting conditions.
- **Dataset B — Pothole, Cracks, and Open Manholes (Sabid Rahman):** A specialised road-hazard dataset focusing on potholes, road cracks, open manholes, curbs, vendors, and sewer covers. Images were captured from pedestrian and vehicular perspectives across multiple cities.

### 3.2 Data Integration via Roboflow

Both datasets were uploaded to a unified Roboflow project (visually\_impaired\_ai\_v2) using the YOLOv8 format. Roboflow performed the following preprocessing and augmentation steps automatically:

- Image resizing to 640×640 pixels
- Automatic train/validation/test split (70% / 20% / 10%)
- Label normalisation and deduplication
- Augmentations: horizontal flips, brightness variation, and mosaic composition

### 3.3 Class Mapping

The combined dataset produces 22 obstacle classes. The following table lists the class IDs and their corresponding labels as defined in the model's prediction dictionary:

| Class ID | Label                  | Class ID | Label              |
|----------|------------------------|----------|--------------------|
| 0        | Pothole                | 11       | Person             |
| 1        | Cracks in the road     | 12       | Fence or Wall      |
| 2        | Open Manhole           | 13       | Barrier            |
| 3        | Vendor                 | 14       | Road Work Obstacle |
| 4        | Curb                   | 15       | Door               |
| 5        | Crack in the Road      | 16       | Tree               |
| 6        | Trash Bin              | 17       | Flower Pot         |
| 7        | Bench                  | 18       | Sewer              |
| 8        | Open Manhole (variant) | 19       | Stairs             |
| 9        | Car                    | 20       | Pole               |
| 10       | Motorcycle             | 21       | Crosswalk          |

### 3.4 Model Training

Training was performed in Google Colab using a GPU-accelerated runtime. The YOLOv8n pre-trained weights (yolov8n.pt) were used as the starting point for transfer learning, allowing the model to leverage general visual features learned from the COCO dataset.

Training configuration:

| Parameter                        | Value                       |
|----------------------------------|-----------------------------|
| Base model                       | YOLOv8n (nano)              |
| Pre-training dataset             | COCO (80 classes)           |
| Training epochs                  | 25                          |
| Image size                       | 640 × 640 px                |
| Confidence threshold (inference) | 0.4                         |
| Optimiser                        | AdamW (Ultralytics default) |
| Hardware                         | Google Colab GPU (T4)       |
| Framework                        | Ultralytics YOLOv8 (Python) |

### 3.5 Model Persistence

After training, the best model weights (best.pt) were automatically saved to runs/detect/train/weights/ by the Ultralytics framework and subsequently backed up to Google Drive (YOLO\_Sensor\_Backup/train/weights/best.pt). This ensured the trained model could be reloaded in future Colab sessions without retraining.

### 3.6 Inference & Validation

Validation was conducted on held-out images from the Roboflow dataset as well as a separate folder of manually curated test images stored in Google Drive (Test\_Images/img-val/). Predictions were run with a confidence threshold of 0.10 for exploratory testing (to reveal low-confidence detections) and 0.40 for production inference.

## 4. System Architecture

### 4.1 Overview

The system follows a client-server architecture. The front-end (HTML/JavaScript) acts as the client, capturing camera frames and displaying detection results. The back-end (Python FastAPI) hosts the YOLOv8 model and performs inference. Communication between the two components uses HTTP POST requests with JPEG image payloads.

### 4.2 Back-End: FastAPI Prediction Server

The API server is built with FastAPI, a high-performance asynchronous Python web framework. It exposes a single POST endpoint at `/predict` that accepts a multipart image file, runs YOLOv8 inference, and returns a JSON array of detections.

#### Key Components

- CORS middleware: allows cross-origin requests from any browser client
- Image loading: `PIL.Image` is used to decode the uploaded JPEG from memory
- Ultralytics YOLO: `model.predict()` runs inference at `conf=0.4`
- Custom label dictionary: maps numeric class IDs to descriptive string labels
- Response format: each detection returns a label and a bounding box in `[x1, y1, x2, y2]` pixel coordinates

#### Response Structure

```
{
  "detections": [
    {
      "label": "pothole",
      "box": [x1, y1, x2, y2]
    },
    ...
  ]
}
```

### 4.3 Tunnel: ngrok

Google Colab runs in an isolated cloud environment with no public IP. The `pyngrok` library establishes a secure HTTPS tunnel from Colab's local port 8000 to a public URL (e.g. `https://purchase-tissue-wrench.ngrok-free.dev`). This URL is injected into the front-end HTML as the `API_URL` constant.

Note: ngrok free-tier URLs are ephemeral — they change every time the Colab session is restarted, requiring a manual URL update in the HTML file.

### 4.4 Front-End: Browser Client

The front-end is a single HTML file with embedded CSS and JavaScript. It performs four core functions:

- Camera access: uses `navigator.mediaDevices.getUserMedia` with `facingMode: 'environment'` to access the rear camera
- Frame capture: a hidden canvas element draws the current video frame and exports it as a JPEG blob every 1 second
- API call: the JPEG blob is sent to the FastAPI endpoint via a multipart `FormData` POST request

- **Result rendering:** an overlay canvas positioned over the video element draws neon-green bounding boxes and labels; the Web Speech API reads out the first detected obstacle class

## Overlay Rendering

The overlay canvas is positioned absolutely over the video element and sized to match the video's intrinsic resolution (`videoWidth` × `videoHeight`) to ensure accurate pixel mapping between the model's output coordinates and the displayed bounding boxes. CSS pointer-events: none ensures touch and click events pass through to the video element.

## 4.5 System Diagram

| Component       | Technology               | Role                                            |
|-----------------|--------------------------|-------------------------------------------------|
| Camera Client   | HTML5 / JavaScript       | Captures frames, displays overlays, plays audio |
| Frame Transport | HTTP POST / FormData     | Sends JPEG frames to back-end API               |
| Tunnel          | ngrok (pyngrok)          | Exposes Colab server to public HTTPS URL        |
| API Server      | Python FastAPI + uvicorn | Receives frames, runs inference, returns JSON   |
| ML Model        | YOLOv8n (Ultralytics)    | Detects obstacles in image frames               |
| GPU Runtime     | Google Colab (T4 GPU)    | Hosts training and inference compute            |
| Model Storage   | Google Drive             | Persists trained weights across sessions        |

## 5. Implementation Details

### 5.1 Development Environment

The project was developed and executed entirely within Google Colab, which provided free access to NVIDIA T4 GPU resources. Python package management was handled via pip within the Colab runtime. The front-end client was developed locally using a text editor and served directly as a static HTML file in the browser.

### 5.2 Dataset Download & Preparation

Datasets were downloaded programmatically using the Kaggle API. The KAGGLE\_API\_TOKEN environment variable was set at runtime to authenticate the download. After downloading, both archives were extracted into separate directories (dataset\_A, dataset\_B) before being uploaded to Roboflow.

```
# Authenticate and download datasets
os.environ['KAGGLE_API_TOKEN'] = '<token>'
!kaggle datasets download -d nayanbebale/obstacles-detection-datasets-yolo
!kaggle datasets download -d sabidrahman/pothole-cracks-and-openmanhole
```

### 5.3 Model Training Code

Transfer learning was applied by loading the pre-trained YOLOv8n weights and fine-tuning on the merged Roboflow dataset:

```
from ultralytics import YOLO

model = YOLO('yolov8n.pt') # Load pre-trained nano weights
results = model.train(
    data=f'{dataset.location}/data.yaml',
    epochs=25,
    imgsz=640,
    plots=True
)
```

### 5.4 FastAPI Prediction Endpoint

The prediction endpoint performs inference on each incoming frame and maps numeric class IDs to human-readable labels using a hand-crafted dictionary derived from the Roboflow project's class list:

```
@app.post('/predict')
async def predict(file: UploadFile = File(...)):
    contents = await file.read()
    img = Image.open(io.BytesIO(contents))
    results = model(img, conf=0.4)
    detections = []
    for box in results[0].boxes:
        class_id = int(box.cls)
        label = CUSTOM_LABELS.get(class_id, f'Unknown ({class_id})')
        coords = box.xyxy[0].tolist()
        detections.append({'label': label, 'box': coords})
    return {'detections': detections}
```

## 5.5 Front-End Frame Capture Loop

The client captures and sends frames at a fixed interval using a recursive `setTimeout` pattern to avoid overlapping requests:

```
async function analyze() {
  // Snapshot current video frame to hidden canvas
  captureCanvas.getContext('2d').drawImage(video, 0, 0);
  captureCanvas.toBlob(async (blob) => {
    const formData = new FormData();
    formData.append('file', blob, 'frame.jpg');
    const response = await fetch(API_URL, { method: 'POST', body: formData });
    const data = await response.json();
    // Draw boxes and trigger speech...
  }, 'image/jpeg', 0.6);
  setTimeout(analyze, 1000); // 1 frame per second
}
```

## 6. Alternatives Considered

### 6.1 Detection Model Alternatives

#### 6.1.1 YOLOv8s / YOLOv8m

The small (s) and medium (m) YOLOv8 variants offer higher mAP but at the cost of inference speed (roughly 2× and 4× slower respectively) and model size. On a free Colab T4 they remain real-time, but they would not run acceptably on a mid-range smartphone CPU without quantisation.

- Why not chosen: The latency budget of 300 ms was tight enough that the nano model was the safer choice for the current client-server architecture; accuracy gains from the larger models were not worth the risk of exceeding the budget.
- When it would be better: If the system were refactored to run on a server with a dedicated GPU (e.g. a cloud VM), the medium variant would be preferable

### 6.2 Training Environment Alternatives

#### 6.2.1 Local Machine / Personal GPU

Training on a local GPU (e.g. an RTX 3060) would allow longer training runs without session timeouts and persistent file storage without Drive. However, not all team members had access to a CUDA-capable GPU.

- Why not chosen: Google Colab provided a free, accessible GPU for the duration of the project without requiring hardware investment.

## 7. Results & Evaluation

### 7.1 Training Performance

The model was trained for 25 epochs on the combined Roboflow dataset. YOLOv8 automatically logs training and validation metrics after each epoch, producing loss curves and precision-recall plots. Key observations during training:

- Box regression loss (train/val) decreased steadily throughout training, indicating the model was learning accurate spatial localisation
- Classification loss showed rapid convergence in the first 10 epochs due to transfer learning from COCO pre-training
- Validation mAP@50 improved consistently without signs of overfitting at epoch 25

### 7.2 Qualitative Validation

Visual validation was performed on held-out images from the Roboflow dataset using multiple confidence thresholds:

- **conf=0.10 (exploratory):** Revealed a broad range of detections including low-confidence candidates, useful for understanding model coverage
- **conf=0.40 (production):** Filtered to high-confidence detections, reducing false positives while maintaining coverage of primary obstacles

Manual inspection of prediction outputs on diverse test images confirmed correct detection of the following classes: potholes, open manholes, poles, motorcycles, persons, curbs, and road cracks.

### 7.3 Real-Time Performance

End-to-end latency (from frame capture to audio alert) was evaluated informally during browser testing:

| Stage                                  | Approx. Latency |
|----------------------------------------|-----------------|
| Frame capture & JPEG encoding (client) | ~30 ms          |
| HTTP transmission to ngrok tunnel      | ~80–150 ms      |
| YOLOv8 inference on Colab GPU          | ~40–80 ms       |
| JSON response + bounding box rendering | ~20 ms          |
| Total end-to-end (1 fps operation)     | ~200–300 ms     |

At a 1-frame-per-second scan rate, the system provides alerts within approximately 300 ms of a hazard appearing in frame — sufficient for walking-pace navigation.

### 7.4 Limitations

- Ephemeral API URL: the ngrok tunnel URL changes on every Colab restart, requiring manual updates to the HTML file
- 1 fps scan rate: suitable for walking pace but insufficient for dynamic fast-moving hazards
- GPU dependency: the FastAPI server requires Colab GPU runtime; CPU-only inference is significantly slower

- Network dependency: the system requires a stable internet connection for client-server communication
- Training epochs: 25 epochs may be insufficient for full convergence on the 22-class dataset; additional training could improve mAP

## 8. Discussion

### 8.1 Design Decisions

The choice of YOLOv8n was motivated by its compact size and fast inference time, which are critical for a system intended to run close to real-time. While larger YOLO variants (YOLOv8s, YOLOv8m) offer higher accuracy, they exceed the latency budget for interactive navigation assistance.

The browser-based front-end was chosen over a native mobile app to maximise accessibility — any smartphone with a modern browser can run the client without installation. The overlay canvas approach avoids the need for server-side image annotation, reducing bandwidth and latency.

Roboflow was selected for dataset management because it handles the fragile task of merging datasets with different annotation formats and label schemas, reducing the risk of format errors that could corrupt training.

### 8.2 Current Limitations

#### 8.2.1 Internet Dependency

The client relies on a stable mobile data or Wi-Fi connection to reach the Colab inference server. In areas with poor connectivity (underground passages, rural environments) the system would fail silently.

#### 8.2.2 1 fps Scan Rate

The current architecture sends one frame per second via sequential HTTP POST requests. At walking pace (~1.4 m/s) a user could travel approximately 1.4 metres between scans, which is sufficient for stationary hazards but inadequate for fast-moving obstacles such as cyclists or motorcycles crossing the path.

#### 8.2.3 Training Depth

25 epochs may be insufficient for full convergence on a 22-class dataset, particularly for classes that are visually similar (e.g. class 2 and class 8 are both open manholes from different datasets) or have few training examples (vendors, flower pots). Extended training would likely improve recall on these minority classes.

### 8.3 Future Work

- On-device inference: export the model to ONNX or TensorFlow Lite for local inference on Android/iOS, eliminating the network dependency and ngrok limitation
- Increased frame rate: optimise the capture-inference-render pipeline to achieve 5–10 fps using a WebSocket stream instead of sequential HTTP requests
- Depth estimation: integrate a monocular depth estimation model to provide distance alerts (e.g. 'pothole 2 metres ahead')
- Additional training: extend training to 50–100 epochs on a larger, more diverse dataset to improve mAP
- Wearable integration: explore mounting options (glasses, chest strap) and Bluetooth earphone pairing for a fully hands-free experience
- User testing: conduct structured usability testing with visually impaired participants to evaluate real-world effectiveness and gather feedback

## 9. Conclusion

This project successfully demonstrates a functional real-time obstacle detection system for visually impaired navigation, built entirely with open-source tools and free cloud resources. By fine-tuning a compact YOLOv8n model on two obstacle-specific datasets and deploying it through a FastAPI server accessible via a browser-based camera client, the system achieves sub-second hazard detection and audio alerting for 22 urban obstacle classes.

The architecture is lightweight, accessible, and extensible. The primary limitations, ephemeral tunnels and network dependency, are addressable through on-device model export, representing a clear pathway to a production-ready assistive device. The project validates the core technical approach and provides a solid foundation for future development toward a deployable assistive navigation tool.

## References

1. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. CVPR 2016.
2. Ultralytics. (2023). YOLOv8: A New State-of-the-Art YOLO Model. <https://github.com/ultralytics/ultralytics>
3. Bebale, N. (2023). Obstacles Detection Datasets (YOLO). Kaggle. <https://www.kaggle.com/datasets/nayanbebale/obstacles-detection-datasets-yolo>
4. Rahman, S. (2023). Pothole, Cracks, and Open Manhole Dataset. Kaggle. <https://www.kaggle.com/datasets/sabidrahman/pothole-cracks-and-openmanhole>
5. Roboflow. (2024). Roboflow Computer Vision Platform. <https://roboflow.com>
6. FastAPI. (2024). FastAPI Framework. <https://fastapi.tiangolo.com>
7. ngrok. (2024). ngrok Secure Tunnels. <https://ngrok.com>
8. World Health Organization. (2023). Blindness and Vision Impairment Fact Sheet. <https://www.who.int/news-room/fact-sheets/detail/blindness-and-visual-impairment>
9. Claude AI (used for structuring the report)